ЗВІТ з лабораторної роботи

Виконала: студентка 3 курсу, групи тк-31, Янчук Дарія

NoSQL у системі управління навчальними курсами

Електронний контент курсів і спроби тестів винесено в MongoDB, той самий фрагмент реалізовано в PostgreSQL, швидкодію порівняно на 2 000 і 20 000 курсів; друга NoSQL-база — Redis.

1. Види NoSQL баз даних

NoSQL-бази відрізняються від реляційних тим, що не використовують жорстку табличну схему і оптимізовані під конкретні задачі. Основні типи:

- **Ключ-значення (Key-value):** Зберігають дані у вигляді словника (Redis, DynamoDB). Дуже швидкі, але не підходять для складних запитів. У поточній системі їх можна використати для кешування сесій.
- **Документні:** Зберігають дані у форматі JSON/BSON (MongoDB, CouchDB). Дозволяють зберігати вкладені об'єкти та масиви. Добре підходять для контенту курсів і збереження спроб тестів.
- **Колонкові:** Оптимізовані для швидкого запису великих обсягів даних (Cassandra). Можна застосувати для логування дій користувачів системи.
- **Графові:** Використовуються для складних зв'язків (Neo4j). Підходять для аналізу пререквізитів дисциплін.

При проєктуванні NoSQL баз схема зазвичай будується відштовхуючись від запитів застосунку, а не від структури сутностей. Це часто вимагає денормалізації даних.

2. Чи бцде ефективніше виносити частину даних в іншу базу

Поточну схему переносити в NoSQL точно не ефективніше: це довідники зі зв'язками, обмеження унікальності й юридично значущі оцінки. Але два фрагменти, які з'являються при розширенні системи до повноцінної LMS, ефективніше тримати в документній БД.

Дані	Характер	Де тримати	Чому
Users, Students, Teachers, Departments, AcademicGroups	Довідники, багато зв'язків, рідкі зміни	PostgreSQL	Зв'язки один-до-одного та FK — сильна сторона SQL; дублювати ПІБ у документах дорого (тест W5)

Дані	Характер	Де тримати	Чому
Curriculums, Schedule, Classrooms	Композитні унікальні ключі, перетини	PostgreSQL	UNIQUE (аудиторія, дата, час) і транзакції не дають накладок у розкладі; у NoSQL це довелося б перевіряти в коді
ExamRecords, Attendance	Офіційні оцінки, аудит, процедури	PostgreSQL	Потрібні ACID, CHECK, логічне видалення й процедури з попередньої лаби
Syllabuses → Topics + електронні матеріали	Глибоке дерево (курс → модулі → матеріали → питання → варіанти), поліморфні матеріали, читається цілком	MongoDB	1 документ замість 12 таблиць; немає NULL-колонок для різних типів; сторінка курсу — 1 запит
Спроби тестів з відповідями	Масовий незмінний запис, читається цілою спробою	MongoDB	Запис спроби — 1 insert замість транзакції в 3 таблиці; пік під час контрольних
Сесії, рейтинги, присутність онлайн, кеш	Тимчасові, «гарячі», прості структури	Redis	TTL, атомарні лічильники, sorted set (розділ 6)

Розширення вимог до застосунку:

Для обґрунтування використання NoSQL систему було розширено модулем «Електронний курс». Вимоги:

1. Кожен силабус містить набір модулів та навчальних матеріалів (лекції, відео, тести).
2. Тести мають питання різного типу і варіанти відповідей.
3. Студенти проходять тести, і система зберігає кожну їхню спробу.

Обґрунтування: Використання документних баз, тобтоMongoDB, для цієї задачі ефективніше, оскільки структура курсу має ієрархічний вигляд і зазвичай читається застосунком цілком. У PostgreSQL для цього довелось би робити складні запити з багатьма JOIN до різних таблиць (матеріали, питання, варіанти). У MongoDB весь курс або уся спроба тесту завантажується як один цілісний документ. Мінусом такого підходу є складність побудови аналітичних звітів по всьому університету та необхідність синхронізації денормалізованих даних.

3. Реалізація в MongoDB

В базі lms створено дві колекції (mongo/setup.js, mongo/indexes.js): course_content — один документ на весь електронний курс силабусу (_id = Syllabuses.Id), і quiz_attempts — один документ на спробу тесту з масивом відповідей. Зв'язок з реляційною частиною тримається на ідентифікаторах.

Скорочена структура документа курсу:

```
{
 "_id": 42,
 "disciplineId": 153, "disciplineCode": "D0153", "disciplineName": "...",
 "teacherId": 38, "teacherName": "Бондаренко Богдан",
 "departmentId": 1, "departmentShortName": "K01",
 "semesterId": 7, "academicYear": "2024/2025", "semesterNo": 1,
 "version": 1, "updatedAt": ISODate("2025-09-08T16:47:46Z"),
 "modules": [
   { "topicId": 6011, "orderNo": 1, "title": "...", "hours": 6,
    "materials": [
      { "materialId": 30154, "orderNo": 1, "type": "lecture", "title": "...",
        "isRequired": true, "tags": ["t003", "t017"], "body": "...", "readingTimeMin": 12 },
      { "materialId": 30155, "type": "video", "url": "https://...", "durationSec": 1820, ... },
      { "materialId": 30156, "type": "quiz", "timeLimitMin": 20, "attemptsAllowed": 2,
        "passingScore": 14,
        "questions": [
          { "questionId": 91001, "orderNo": 1, "text": "...?", "qtype": "single", "points": 3,
            "options": [ { "optionId": 330001, "orderNo": 1, "text": "...", "isCorrect": false } ] } ] },
      { "materialId": 30157, "type": "assignment", "deadline": ISODate("..."), "maxPoints": 18,
        "rubric": [ { "criterionId": 5001, "orderNo": 1, "criterion": "...", "points": 6 } ] }
   ] }
 ]
}
```

Проєктні рішення:

- **Вкладення замість посилань.** Модулі, матеріали, питання, варіанти і рубрики читаються разом із курсом і не існують без нього, тому межа агрегату — курс.
- **Поліморфізм.** Поля підтипу лежать на рівні матеріалу. $jsonSchema з oneOf вимагає, наприклад, questions і passingScore для type: "quiz"; документ без них відхиляється.
- **Денормалізація заголовка.** disciplineName, teacherName, departmentId скопійовано з PostgreSQL, щоб сторінка курсу і звіти не зверталися до SQL. Ціна — синхронізація при зміні (тест W5).
- **Спроби — окрема колекція, а не масив у курсі.** Їх кількість необмежена (ліміт 16 МБ на документ, конкуренція записів у один документ). У спробі денормалізовано quizTitle і syllabusId для історії студента.
- **Ідентифікатори вкладених елементів** (materialId, questionId, optionId) збережено: на них посилаються спроби і оновлення через arrayFilters.

Індекси: {departmentId:1, semesterId:1}, {teacherId:1}, multikey {"modules.materials.tags":1} і {"modules.materials.materialId":1}; для спроб — {studentId:1, finishedAt:-1} і {quizId:1}. Приклади запитів (CRUD, $push/$pull, arrayFilters, aggregation, explain) — у mongo/queries.js.

4. Той самий фрагмент у PostgreSQL

Дві колекції MongoDB відображаються в 15 таблиць PostgreSQL (sql/02_course_content.sql). Поліморфні матеріали змодельовано патерном «таблиця на підклас» (class table inheritance): спільні поля в Materials, специфічні — у таблицях 1:1.

Рівень документа MongoDB	Таблиці PostgreSQL
course_content (заголовок)	Syllabuses (+ ContentVersion, ContentUpdatedAt), JOIN Disciplines, Teachers, Users, Departments, Semesters
modules[]	Topics (+ OrderNo)
modules[].materials[]	Materials
поля лекції / відео / файлу / посилання	MaterialLectures, MaterialVideos, MaterialFiles, MaterialLinks
поля тесту	Quizzes
questions[]	QuizQuestions
questions[].options[]	QuizOptions
поля завдання; rubric[]	Assignments; AssignmentRubric
tags[]	MaterialTags
quiz_attempts	QuizAttempts
answers[]	AttemptAnswers
answers[].selectedOptionIds[]	AttemptAnswerSelections

Кожному індексу MongoDB створено еквівалент (sql/03_indexes.sql), а також індекси на всі FK. Один з них з'явився після першого прогону: без індексу на AttemptAnswerSelections.OptionId каскадне видалення варіантів відповідей сканувало всю таблицю для кожного варіанта, і заміна курсу тривала 1,6 с замість 11 мс. У MongoDB такої пастки немає, бо немає FK.

5. Методика тестування швидкодії

Мета — порівняти дві БД на тих операціях, які реально робитиме застосунок, і перевірити, де документна модель дає виграш, а де програє.

Дані. Генератор (lab/datagen.py) створює кожен курс один раз і записує його в обидві БД, тому вміст баз однаковий. Два обсяги:

Обсяг	Курсів	Матеріалів	Питань у тестах	Спроб тестів
S	2 000	90 571	96 106	41 715
L	20 000	898 905	946 177	401 685

L — це університет на ~9 000 студентів і 600 викладачів з курсами за 5 років. Два обсяги потрібні, щоб побачити, чи змінюється результат з ростом даних.

Перевірка. Перед тестами lab/verify.py перевіряє, що обидві БД повертають однакові результати (40 курсів, 30 тестів, пошук за тегами, звіт). Так ми порівнюємо запити, які роблять одне й те саме.

Як мірялось:

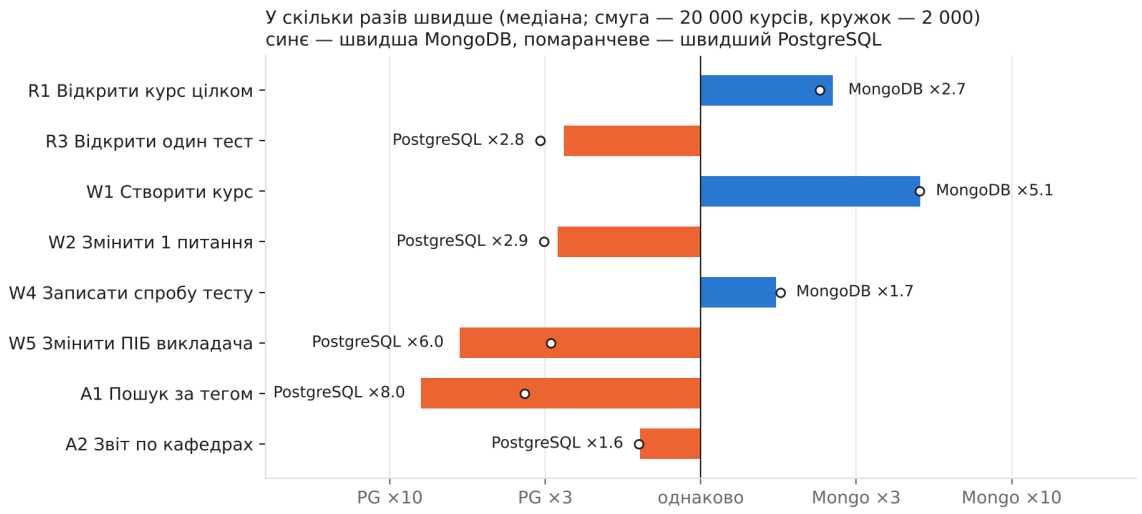
1. Однакові індекси в обох БД.
2. У час входить не лише запит, а й перетворення результату в об'єкт, який потрібен програмі. Для SQL це включає збирання дерева курсу з рядків.
3. Перед заміром — прогрів (кілька запитів, які не рахуються).
4. Кожен тест повторюється в 3 раунди, і в кожному раунді БД запускаються в іншому порядку, щоб жодна не мала переваги.
5. Id курсів і тестів вибираються випадково, але однакові для обох БД.
6. Запис з однаковою надійністю: обидві БД підтверджують запис лише після запису журналу на диск (MongoDB j:true, PostgreSQL synchronous_commit = on).
7. Результат — медіана (типовий час) і p95 (час, який не перевищують 95% запитів).

Сценарії і очікування:

Код	Що робимо	Очікування
R1	Відкрити курс цілком	MongoDB швидша: 1 документ проти 12 таблиць
R3	Відкрити один тест	PostgreSQL швидший: MongoDB читає весь курс заради одного тесту
W1	Створити курс	MongoDB швидша: 1 вставка проти сотень рядків у 12 таблицях
W2	Змінити бали одного питання	PostgreSQL швидший: міняється 1 рядок
W4	Записати спробу тесту	MongoDB швидша: 1 документ проти 3 таблиць
W5	Змінити ПІБ викладача	PostgreSQL швидший: у MongoDB ПІБ скопійовано в кожен його курс
A1	Пошук матеріалів за тегом у всіх курсах	PostgreSQL швидший: MongoDB розбирає кожен знайдений курс цілком
A2	Звіт по кафедрах	PostgreSQL швидший: аналітика по всіх даних

Результати

Тести запускались на машині з 2 ядрами та 7 ГБ оперативної пам'яті: PostgreSQL 16, MongoDB 8.3, програма тестування — на Python 3.13. Загальний підсумок: MongoDB швидша, коли треба прочитати або записати курс чи спробу цілком. PostgreSQL швидший, коли треба дістати чи змінити невелику частину даних або порахувати щось по всіх курсах.



Результати для 20 000 курсів (медіана, в дужках p95), мс:

Сценарій	MongoDB	PostgreSQL	Хто швидший
R1 Відкрити курс цілком	0,92 (1,35)	2,45 (4,25)	MongoDB ×2,7
R3 Відкрити один тест	0,91 (1,24)	0,33 (0,65)	PostgreSQL ×2,8
W1 Створити курс	2,27 (3,34)	11,5 (17,7)	MongoDB ×5,1
W2 Змінити бали одного питання	1,78 (3,02)	0,62 (1,13)	PostgreSQL ×2,9
W4 Записати спробу тесту	0,62 (1,13)	1,09 (1,90)	MongoDB ×1,7
W5 Змінити ПІБ викладача	2,21 (26,1)	0,37 (0,69)	PostgreSQL ×6,0
A1 Пошук за тегом у всіх курсах	41,9 (49,0)	5,3 (6,7)	PostgreSQL ×8,0
A2 Звіт по кафедрах	2 178 (2 466)	1 391 (1 425)	PostgreSQL ×1,6

Читання

Щоб відкрити курс (R1), MongoDB читає один документ. PostgreSQL мусить зібрати курс із 12 таблиць. Для PostgreSQL випробувано два способи: один великий запит з JOIN (2,79 мс) і 6 простих запитів (2,45 мс). В таблиці наведено швидший. Один великий JOIN повільніший, бо повторює дані матеріалу в кожному рядку з варіантом відповіді.

Щоб відкрити один тест (R3), MongoDB все одно читає весь курс (~70 КБ), бо тест лежить усередині документа. PostgreSQL читає лише потрібні рядки з 3 таблиць.

Запис

Створення курсу (W1) у PostgreSQL — це вставка ~400 рядків у 12 таблиць з перевіркою всіх зовнішніх ключів, а в MongoDB — один документ. Запис спроби (W4) — та сама ситуація в меншому масштабі: 3 таблиці проти 1 документа.

Зміна одного питання (W2) показує зворотний бік: MongoDB шукає питання всередині вкладених масивів (arrayFilters) і перезаписує документ, а PostgreSQL оновлює один рядок за ключем.

Зміна ПІБ викладача (W5) показує ціну дублювання даних. У MongoDB ПІБ скопійовано в кожен курс викладача (в середньому 33 курси), і їх усі треба оновити. У PostgreSQL це один рядок у Users. Особливо видно по p95: 26 мс проти 0,7 мс.

Аналітика

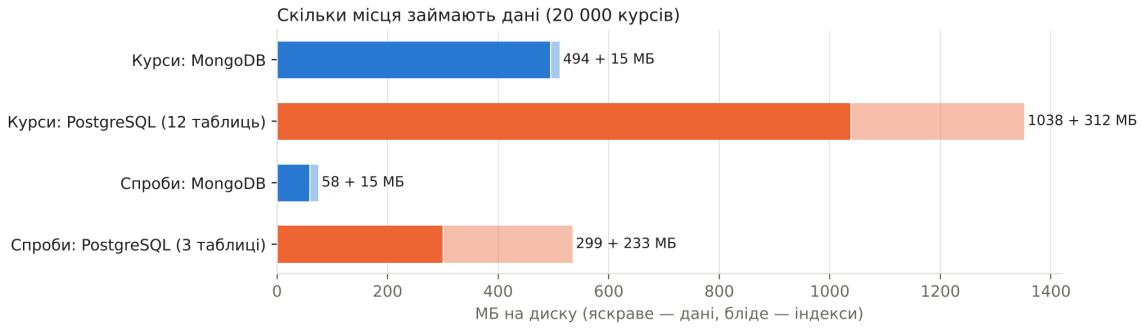
Пошук за тегом (A1) і звіт по кафедрах (A2) виграє PostgreSQL. Індекс MongoDB швидко знаходить потрібні курси, але потім кожен курс доводиться розгортати цілком ($unwind), щоб відібрати кілька матеріалів. PostgreSQL одразу читає лише потрібні рядки з таблиці тегів.

2 000 проти 20 000 курсів

Запити до одного курсу чи тесту (R1, R3, W1, W2, W4) майже не сповільнились, бо індекс знаходить запис за однаковий час. Запити по всіх курсах (A1, A2) сповільнились у 4–15 разів, бо даних стало в 10 разів більше. Хто швидший, не змінилось в жодному сценарії.

Розмір на диску

Курси в MongoDB займають 509 МБ проти 1 350 МБ у PostgreSQL, спроби — 73 МБ проти 532 МБ. MongoDB стискає дані на диску, а в PostgreSQL кожен з мільйонів рядків має свій службовий заголовок і відомості в індексах.



Висновки

Висновки Під час виконання лабораторної роботи було досліджено принципи роботи з NoSQL базами даних на прикладі MongoDB та Redis. Було реалізовано паралельну структуру збереження електронних курсів у PostgreSQL та MongoDB для порівняння їхньої швидкодії.

За результатами тестування встановлено наступне:

1. Операції читання та запису цілих агрегатів виконуються швидше в MongoDB.
2. Точкові оновлення даних та аналітичні запити по всій базі швидше виконуються в PostgreSQL.
3. Загальний обсяг бази на диску у випадку MongoDB виявився меншим через відсутність великої кількості таблиць та індексів для зовнішніх ключів.

Отже, для розв'язання нішевих задач зі складною ієрархічною структурою даних доцільно використовувати документні бази даних. При цьому основні довідкові та транзакційні дані системи ефективніше залишати у реляційній БД.

6. Redis — NoSQL типу key-value

Redis — сховище «ключ → значення» в оперативній пам'яті, де значенням може бути не лише рядок, а й структура даних з атомарними операціями. Запитів «за значенням» немає: ключі проєктуються так, щоб застосунок завжди знав їх заздалегідь (session:<token>, leaderboard:quiz:<id>). Повний протокол з відповідями сервера — results/redis_demo.txt, код — lab/redis_demo.py.

Тип даних	Задача в системі	Команди
STRING + TTL	Сесії користувачів, автоматичний вихід	SET session:a1b2 user:15 EX 1800 NX, GET, TTL, EXPIRE, DEL
Лічильники	Перегляди курсу, обмеження спроб входу	INCR, INCRBY, INCR + EXPIRE
HASH	Картка студента	HSET, HGET, HMGET, HINCRBYFLOAT, HGETALL, HDEL
LIST	Стрічка сповіщень, черга перевірки робіт	LPUSH, LTRIM, LRANGE, RPUSH, LPOP, BLPOP
SET	Хто присутній на занятті, хто відсутній	SADD, SISMEMBER, SCARD, SINTER, SDIFF, SUNION
SORTED SET	Рейтинг студентів за тестом	ZADD, ZINCRBY, ZREVRANGE … WITHSCORES, ZREVRANK, ZRANGEBYSCORE, ZCOUNT
Транзакція	Зарахування спроби однією дією	MULTI, ZADD, INCR, LPUSH, EXEC
Службові	Пошук ключів, тип значення	SCAN 0 MATCH leaderboard:*, TYPE, DBSIZE

Фрагмент протоколу, рейтинг за тестом:

```
> ZINCRBY leaderboard:quiz:12345 3 student:15        # перерахунок після апеляції
  16.0
> ZREVRANGE leaderboard:quiz:12345 0 4 WITHSCORES    # топ-5
  [['student:23', 30.0], ['student:12', 30.0], ['student:18', 29.0], ['student:13', 29.0], ['student:21', 25.0]]
> ZREVRANK leaderboard:quiz:12345 student:15         # місце студента (з 0)
  10
> ZRANGEBYSCORE leaderboard:quiz:12345 -inf (18      # не склали (поріг 18)
  ['student:14', 'student:16', 'student:10', 'student:24', 'student:15', 'student:22']
> SINTER online:lesson:1001 online:lesson:1002       # були на обох заняттях
  {'15', '17'}
> BITOP AND attendance:both attendance:lesson:1001 attendance:lesson:1002
> BITCOUNT attendance:both                          # були на обох
  4
```

У таких задачах Redis замінює цілі таблиці: рейтинг у SQL вимагав би ORDER BY score з віконною функцією для місця при кожному запиті, а ZREVRANK відповідає за O(log N); сесія з TTL зникає сама, без фонового очищення.

Файли проєкту

sql/ — таблиці PostgreSQL; mongo/ — колекції MongoDB і приклади запитів; lab/ — генератор даних, запити, перевірка, тести швидкості, Redis; results/ — результати і графіки. Як запустити — у README.md.